

One-machine Scheduling for Minimizing Total Flow Time with Release Dates

Chengbin CHU

*Projet SAGEP, INRIA-Lorraine, CESCO, Technopôle Metz 2000
4 rue Marconi, 57070 METZ, FRANCE*

ABSTRACT

In this paper, we deal with the one-machine scheduling problem to minimize total flow time with different release dates. This problem is equivalent to the problems of minimizing total completion time, total job lateness, and mean number of jobs in the shop. This problem is NP-hard.

We prove a necessary and sufficient condition for local optimality which can also be considered as a priority rule. Based on this condition, we define a subset containing all the optimal schedules. Any schedule in this subset verifies some dominance properties proved by other researchers. We also propose some efficient heuristic algorithms using the proven condition to build a schedule belonging to this subset. The algorithm performances are provided.

1. Introduction

The single machine scheduling problem has been extensively studied under different assumptions and various objective functions. In [8], the authors reviewed various results related to this problem. In this paper, we consider the n -job, one machine scheduling problem, under the assumption that the machine performs at most one job at a time and no preemption is allowed. Each job i has a release date r_i and a processing time p_i . The objective function is to minimize the total flow time (i.e. the period between the release date and the completion time). We therefore have the following formulation:

$$\text{Min} \sum_{i=1}^n F_i = \text{Min} \sum_{i=1}^n (C_i - r_i)$$

subject to

$$\begin{cases} \forall i, C_i - p_i \geq r_i \\ \forall i, \forall j, \text{ and } i \neq j, [C_i - p_i, C_i) \cap [C_j - p_j, C_j) = \emptyset \end{cases}$$

where F_i , C_i are respectively the flow time and the completion time of job i corresponding to a schedule. In the literature, this problem is denoted by $n/1/r_i \geq 0/\sum F_i$.

This problem is equivalent to the problems of minimizing the total job lateness, total completion time, total waiting time, mean number of jobs in the shop ([5]).

In the case of identical release dates, this problem can easily be solved by applying SPT (Shortest Processing Time) priority rule ([14]). However, it is NP-hard with different release dates ([10], [13]), which indicates that the existence of a polynomially bounded algorithm is unlikely. Several researchers have proved some dominance properties in different cases (a schedule S dominates another schedule S' if and only if it gives a criterion value at least as good as the one given by S'). A dominance property is a property indicating that some schedules dominate some others. A dominant subset therefore is a subset containing at least one optimal schedule). Chandra ([4]) discussed two models. The first one is the model discussed in this paper, the second one is with pre-emptive-repeat (an operation can be preempted, but the work done before the preemption is lost. If it is processed later, the processing needs a time equal to its initial processing time). Dessouky et al. ([7]) have proved several dominance properties for the problem discussed in this paper. Deogun ([6]) has proposed a partitioning method using these dominance theorems. For the total weighted completion time criterion, Bianco et al. ([2]) have proved several dominance properties, Hariri et al. ([9]) proposed an approach using the relaxation method to obtain a lower bound in a branch and bound (BAB) enumerative algorithm. Posner ([12]) considered the problem with clustered jobs. Chand et al. ([4]) investigated the problem with maximum allowable tardiness constraints.

In this paper, we prove a necessary and sufficient condition for local optimality. By "local optimality", we mean the optimality of two adjacent jobs in a given schedule, without taking into account the jobs following them, assuming that the jobs preceding them are fixed. This is similar to considering the one machine-two jobs problem. This condition can also be considered as a priority rule. Based on this condition, we define a subset, called F-active subset, containing all the optimal schedules. We prove that any schedule in this subset also verifies some dominance properties proved by Dessouky et al. ([7]). We also propose some heuristic algorithms to build a F-active schedule, and we compare their performance with some other priority rules and a BAB algorithm.

In Section 2, we prove the necessary and sufficient condition for local optimality and define the F-active schedule subset. Section 3 presents several heuristic algorithms. The numerical results are given in Section 4.

2. Necessary and sufficient condition for local optimality and F-active schedules

As described in the introduction, obtaining a condition for local optimality is equivalent to considering a two jobs scheduling problem. Let us consider this problem. We have to schedule two jobs i and j , on a machine available after time Δ . We search the condition leading to an optimal schedule.

Assuming that $F_{uv}(\Delta)$ is the sum of flow times of jobs u and v obtained by scheduling u before v (in order to simplify the notations, we note F_{uv} when there is no ambiguity), the problem is to find the condition which leads to $F_{ij} \leq F_{ji}$.

In order to simplify the explanation, we denote

$R_i(\Delta) = \max(\Delta, r_i)$: the earliest beginning time of job i at time Δ ;

and $C_i(\Delta) = R_i(\Delta) + p_i$: the earliest completion time of job i at time Δ .

We will show that the condition we are looking for is obtained using a new dynamic priority measure called PRTF (Priority Rule for Total Flow-time).

Definition 1:

We define a function $PRTF(i, \Delta)$ of a job i at the time Δ as: $PRTF(i, \Delta) = 2R_i(\Delta) + p_i = R_i(\Delta) + C_i(\Delta)$

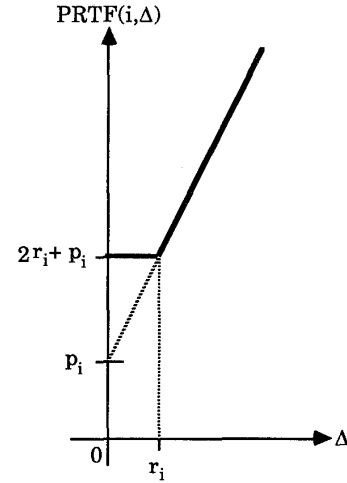


Fig.1 PRTF function

This function only depends on the parameters defining job i . This is a nondecreasing piecewise continuous function (see Fig.1). The function value is, either the sum of the processing time and two times the earliest beginning time of the job at time Δ , or the sum of the earliest beginning time and the earliest completion time of the job at Δ .

The necessary and sufficient condition for local optimality can be obtained according to this function.

Theorem 1:

Consider a scheduling problem involving two jobs i and j which have to be scheduled on a machine available after time Δ . Scheduling i before j is optimal (i.e. the total flow time is minimal) if and only if $PRTF(i, \Delta) \leq PRTF(j, \Delta)$. In other words:

$$PRTF(i, \Delta) \leq PRTF(j, \Delta) \Leftrightarrow F_{ij} \leq F_{ji}$$

Proof:

According to the definitions and the constraints defined in Section 1, we have:

$$\begin{aligned} F_{ij} &= C_i(\Delta) - r_i + \max[C_i(\Delta), r_j] + p_j - r_j \\ &= R_i(\Delta) + \max[C_i(\Delta), \Delta, r_j] + p_i + p_j - (r_i + r_j) \\ &= R_i(\Delta) + \max[C_i(\Delta), R_j(\Delta)] + p_i + p_j - (r_i + r_j) \\ &= \max[R_i(\Delta) + C_i(\Delta), R_i(\Delta) + R_j(\Delta)] + p_i + p_j - (r_i + r_j) \end{aligned}$$

$$\begin{aligned}
&= \max[\text{PRTF}(i, \Delta), Q_{ij}(\Delta)] + P_{ij} - R_{ij} \\
\text{where } Q_{ij}(\Delta) &= R_i(\Delta) + R_j(\Delta) \\
P_{ij} &= p_i + p_j \\
R_{ij} &= r_i + r_j
\end{aligned}$$

In a symmetric way, we can obtain:

$$\begin{aligned}
F_{ji} &= \max[\text{PRTF}(j, \Delta), Q_{ji}(\Delta)] + P_{ji} - R_{ji} \\
&= \max[\text{PRTF}(j, \Delta), Q_{ij}(\Delta)] + P_{ij} - R_{ij}
\end{aligned}$$

Sufficiency is straightforward. Let us now prove the necessity. For this purpose, we have to distinguish three cases.

Case 1° $r_j \geq C_i(\Delta)$

In this case, we have:

$$\begin{aligned}
&R_j(\Delta) \geq C_i(\Delta) \\
\Rightarrow 2R_j(\Delta) + p_j &\geq 2R_i(\Delta) + 2p_i + p_j > 2R_i(\Delta) + p_i \\
\Rightarrow \text{PRTF}(i, \Delta) &\leq \text{PRTF}(j, \Delta)
\end{aligned}$$

Case 2° $r_j < C_i(\Delta)$ and $r_i < C_j(\Delta)$

In this case, we have:

$$\begin{aligned}
F_{ij} &= 2R_i(\Delta) + p_i + p_j - (r_i + r_j) = \text{PRTF}(i, \Delta) + P_{ij} - R_{ij} \\
F_{ji} &= 2R_j(\Delta) + p_j + p_i - (r_i + r_j) = \text{PRTF}(j, \Delta) + P_{ij} - R_{ij} \\
F_{ij} \leq F_{ji} &\Rightarrow \text{PRTF}(i, \Delta) \leq \text{PRTF}(j, \Delta)
\end{aligned}$$

Case 3° $r_i \geq C_j(\Delta)$

In this case, we have:

$$\begin{aligned}
F_{ij} &= 2r_i + 2p_i + p_j - (r_i + r_j) \\
F_{ji} &= 2C_j(\Delta) + p_i - (r_i + r_j) \\
F_{ij} &\geq 2C_j(\Delta) + 2p_i + p_j - (r_i + r_j) \\
&> 2C_j(\Delta) + p_i - (r_i + r_j) = F_{ji}
\end{aligned}$$

In that case, condition $F_{ij} \leq F_{ji}$ does not hold.

Q.E.D.

According to Theorem 1, we can consider the PRTF function as a dynamic priority rule: the smaller the value of PRTF, the higher the priority of the job. An analytic analysis of the PRTF function can be found in the appendix.

This necessary and sufficient condition permits us to build a dominant subset to minimize $\sum F_i$ (i.e. the total flow time). In order to simplify notations, we denote by Δ_i the completion time of the job immediately preceding job i in a schedule. If i is the first job of the schedule, $\Delta_i = -\infty$.

Definition 2:

Consider a given active schedule ⁽¹⁾ S . If any couple of adjacent jobs i and j (i followed by j) verifies at least one of the following conditions:

- 1° $R_i(\Delta_i) < R_j(\Delta_j)$
- 2° $\text{PRTF}(i, \Delta_i) \leq \text{PRTF}(j, \Delta_j)$

S is said to be F-active.

According to the previous definition, Theorem 1 leads to the following corollary.

Corollary 1:

All the optimal schedules of a $n/1/r_i \geq 0/\sum F_i$ scheduling problem are F-active.

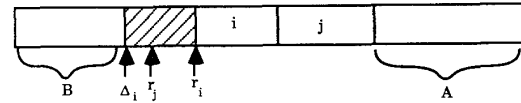
Proof:

Let us assume that there exists an optimal schedule S which is not F-active. There is certainly at least one couple of adjacent jobs i and j (i followed by j) verifying none of the two conditions described in the definition of F-active schedules, then

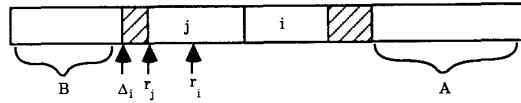
$$\begin{aligned}
R_i(\Delta_i) &\geq R_j(\Delta_j) \\
\text{PRTF}(i, \Delta_i) &> \text{PRTF}(j, \Delta_j)
\end{aligned}$$

Let B be the set of jobs preceding i and A the set of jobs following j . In that case, the total flow time for S is:

$$F = \sum_{l=1}^n F_l = \sum_{l \in B} F_l + \sum_{l \in A} F_l + F_{ij}$$



Schedule S



Schedule S'

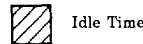


Fig. 2: illustration of corollary 1

¹ An active schedule is a schedule such that no job can be scheduled earlier without delaying another.

We construct another schedule S' by permuting i and j (see Fig. 2). Because $R_i(\Delta_i) \geq R_j(\Delta_i)$, S' is feasible. We then have the total flow time for S'

$$F' = \sum_{l=1}^n F'_l = \sum_{l \in B} F'_l + \sum_{l \in A} F'_l + F_{ji} = \sum_{l \in B} F_l + \sum_{l \in A} F_l + F_{ji}$$

The relation $\text{PRTF}(i, \Delta_i) > \text{PRTF}(j, \Delta_i)$ and Theorem 1 lead to: $F_{ij} > F_{ji}$

We can immediately derive:
 $F > F'$

This contradicts the assumption that S is an optimal schedule.

Q.E.D.

Remarks:

1° The function $\text{PRTF}(i, \Delta)$ is in fact a combination of rules FIFO (First In First Out) and SPT (Shortest Processing Time).

2° If $\forall i, r_i = 0$, the sufficient and necessary condition becomes:

$$p_i \leq p_j$$

It is exactly the SPT rule which leads to an optimal schedule in this case.

3° If $\forall i, p_i = D$, D being a constant, the condition $\text{PRTF}(i, \Delta) \leq \text{PRTF}(j, \Delta)$ becomes

$$R_i(\Delta) \leq R_j(\Delta)$$

which is equivalent to a dynamic FIFO rule. In that case, it leads to an optimal solution.

Properties of F-active schedules

We now establish some properties of F-active schedules. These properties show that any F-active schedule verifies some theorems proved by Dessouky et al. ([7]).

Property 1:

For any job x , if A is the set of jobs following x in a F-active schedule S , then there is no job $y \in A$ of S such that:

$$\forall i \in A \cup \{x\} - \{y\}, p_y < p_i \text{ and } R_x(\Delta_x) \geq R_y(\Delta_x)$$

Proof:

Suppose that there is such a job y in S .

Consider the job w immediately preceding y . Since S is F-active, we have at least one of the following relations:

$$\text{either } R_w(\Delta_w) < R_y(\Delta_w)$$

$$\text{or } 2R_w(\Delta_w) + p_w \leq 2R_y(\Delta_w) + p_y$$

To prove the property, we consider two cases:

$$1^\circ R_w(\Delta_w) < R_y(\Delta_w)$$

In that case, we have $r_y > R_w(\Delta_w) \geq R_x(\Delta_x)$ ⁽²⁾, which implies: $R_y(\Delta_x) > R_x(\Delta_x)$

This contradicts the assumption that $R_y(\Delta_x) \leq R_x(\Delta_x)$

$$2^\circ R_w(\Delta_w) \geq R_y(\Delta_w) \text{ and}$$

$$2R_w(\Delta_w) + p_w \leq 2R_y(\Delta_w) + p_y$$

In this case we have $p_w \leq p_y$, which contradicts the property:

$$\forall i \in A \cup \{x\} - \{y\}, p_y < p_i$$

It therefore is impossible that there exists such a job y in schedule S .

Q.E.D.

Property 2:

For any job x , if A is the set of jobs following x in a schedule S , then there is no job $y \in A$ of S such that:

$$r_x \geq C_y(\Delta_x)$$

Proof:

If there was such a job y in S , S would not be active, hence not F-active either. This contradicts the fact that S is a F-active schedule.

Q.E.D.

These two properties are dominance theorems 1 and 2 proved by Dessouky et al. ([7]), but none of their theorems can decide which job to schedule when there are two jobs to be scheduled.

² The second inequality can be obtained by considering two cases:

a) $w=x$, i.e. x immediately precedes y , then we have

$$R_w(\Delta_w) = R_x(\Delta_x)$$

b) $w \neq x$, w certainly follows x , thus the following relation holds

$$\Delta_w \geq C_x(\Delta_x) = R_x(\Delta_x) + p_x > R_x(\Delta_x)$$

which implies: $R_w(\Delta_w) > R_x(\Delta_x)$

More generally, $R_w(\Delta_w) \geq R_x(\Delta_x)$ holds.

3. Algorithms

According to Corollary 1, we must only consider F-active schedules in order to reach to an optimal solution. Now, we propose some algorithms to construct a F-active schedule.

Since the problem considered in this paper is NP-hard, it is preferable to use heuristic algorithms. For this purpose, the researchers proposed various priority rules which can be easily implemented. In ([11]), readers can find more than one hundred rules used in the scheduling domain.

In [7], Dessouky et al. examined the behaviors of the ECT (Earliest Completion Time) priority rule. They said: "The mean deviation from optimality of the initial ECT sequence was 3% for all problems. Thus, the use of ECT sequencing provides fairly tight upper bounds which can help to eliminate many dominated nodes". In [4], Chandra compared SPT (named EST—Earliest Start Time in [7], the name also used in this paper) with an optimal solution. He concluded that EST gives a solution close to the optimum.

In this section, we build some heuristic algorithms: PRTF (see paragraph 3.2), APRTF (see paragraph 3.3) To evaluate their performance, we compare them with the ECT and SPT rules and the a BAB exact algorithm. The numerical results are provided in Section 4.

All the algorithms definitively schedule a job behind the partial schedule composed of scheduled jobs. At each iteration, the machine begins to be available at time Δ , and there is a set of unscheduled jobs, A . Δ and A are respectively initialized to $-\infty$ and $\{1, 2, \dots, n\}$.

3.1. ECT and EST algorithms

The ECT algorithm: Select job i with $\min_{i \in A} C_i(\Delta)$. Break ties by choosing i with $\min[R_i(\Delta)]$, and further ties by choosing i with $\min(i)$. Schedule i . Update Δ and A , such that $\Delta := C_i(\Delta)$, $A := A - \{i\}$.

The EST algorithm: Select job i with $\min_{i \in A} R_i(\Delta)$. Break ties by choosing i with $\min[p_i]$, and further ties by choosing i with $\min(i)$. Schedule i . Update Δ and A , such that $\Delta := C_i(\Delta)$, $A := A - \{i\}$.

3.2 Algorithm PRTF

Select job i with $\min_{i \in A} \text{PRTF}(i, \Delta)$. Break ties by choosing i with $\min[R_i(\Delta)]$, and further ties by choosing i with $\min(i)$. Schedule i . Update Δ and A , such that $\Delta := C_i(\Delta)$, $A := A - \{i\}$.

3.3 Algorithm APRTF

Step 1: Select job α with $\min_{\alpha \in A} \text{PRTF}(\alpha, \Delta)$. Break ties by choosing α with $\min[R_\alpha(\Delta)]$, and further ties by choosing α with $\min(\alpha)$.

Step 2: Select job β with $\min_{\beta \in A} R_\beta(\Delta)$. Break ties by choosing β with $\min[p_\beta]$, and further ties by choosing β with $\min(\beta)$.

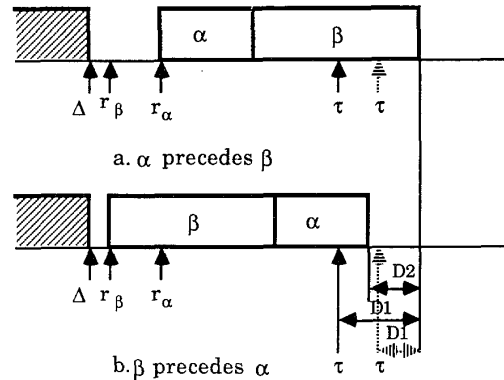
Step 3: If $r_\alpha \leq R_\beta(\Delta)$ then $x = \alpha$. **Goto Step 5.**

Step 4: Noting $\mu = \text{card}(A - \{\alpha, \beta\})$, τ the smallest release date of jobs of $A - \{\alpha, \beta\}$, if the condition $F_{\beta\alpha}(\Delta) - F_{\alpha\beta}(\Delta) < \mu \cdot \min[R_\alpha(\Delta) - R_\beta(\Delta), C_\beta(C_\alpha(\Delta)) - \tau]$ holds, then $x = \beta$, otherwise, $x = \alpha$. **Goto Step 5.**

Step 5: Schedule x . Δ is increased to $C_x(\Delta)$, $A = A - \{x\}$.

Comment:

In fact, at step 4, the algorithm chooses a job to schedule by comparing the gain and the loss. If we scheduled β at time Δ , there would not be idle time on the machine. With α scheduled at that time, we respect the priority rule PRTF (See Fig. 3).



$$D2 = R_\alpha(\Delta) - R_\beta(\Delta)$$

$$D1 = C_\beta(C_\alpha(\Delta)) - \tau$$

Fig. 3 Gain and loss comparison

If we schedule α before β , we can obtain a gain of $F_{\beta\alpha}(\Delta) - F_{\alpha\beta}(\Delta)$, but we create an idle time of value $R_{\alpha}(\Delta) - R_{\beta}(\Delta)$. The unscheduled jobs other than α and β are delayed for at most $\min[R_{\alpha}(\Delta) - R_{\beta}(\Delta), C_{\beta}(C_{\alpha}(\Delta)) - \tau]$. The loss in the worst case then is given by $\mu \cdot \min[R_{\alpha}(\Delta) - R_{\beta}(\Delta), C_{\beta}(C_{\alpha}(\Delta)) - \tau]$. If the gain is at least as great as the loss, we schedule α , otherwise, β is scheduled.

4. Numerical results

In this section, we give some numerical results in order to evaluate the algorithm performances.

We randomly generated 6 samples of examples with respectively 20, 30, 40, 50, 60, 70 jobs. The examples were generated in the same way as in [9]. The processing times were generated between 1 and 100. The release dates were generated between 0 and 50.5. $n \cdot \alpha$. 10 values of α (0.2, 0.4, 0.6, 0.8, 1.0, 1.25, 1.50, 1.75, 2.0, 3.0) permitted us to generate 10 examples for each sample.

Table 1 mean criterion value for each sample

n	20	30	40	50	60	70
APRTF	3995.2	6529.0	10131.8	17048.1	21394.2	25505.7
PRTF	4042.8	6613.2	10229.8	17439.7	22152.2	26366.2
EST	4006.3	6543.0	10141.3	17056.7	21412.1	25535.0
ECT	4419.5	7317.6	11586.6	19293.8	25252.5	29749.4
UPRTF	3995.2	6521.4	10094.9	17042.7	21383.8	25495.1
UET	4006.3	6536.1	10141.3	17056.7	21412.1	25535.0
BAB	3992.2	6475.4	10047.7	16966.6	21346.1	25385.7

UPRTF: a "macro" algorithm consisting in always taking the best solution given by APRTF and PRTF
UET: a "macro" algorithm consisting in always taking the best solution given by EST and ECT

Table 1 shows the mean criterion value given by various heuristics for each sample. From the table, we see that on the average, the best solution found by algorithms APRTF and PRTF is close to the optimum. The deviation is never greater than 0.71%. We also see that for each sample, the best solution found by algorithms APRTF and PRTF (namely, the solution found by UPRTF) is better than the best one found by EST and ECT (namely the one found by UET). The difference between the solution given by UPRTF and

an optimal one is smaller than a half of that between the solution given by UET and an optimal one.

Table 2 mean computation time (in CPU seconds)

n	20	30	40	50	60	70
APRTF	0.0244	0.0561	0.1261	0.1692	0.2476	0.3325
PRTF	0.0144	0.0361	0.0643	0.0894	0.1211	0.1727
EST	0.0112	0.0327	0.0509	0.0728	0.1093	0.1627
ECT	0.0194	0.0378	0.0676	0.0925	0.1360	0.1844
UPRTF	0.0388	0.0922	0.1904	0.2586	0.3687	0.5052
UET	0.0306	0.0705	0.1185	0.1653	0.2453	0.3471
BAB	0.326	2.001	14.73	39.08	290.9	363.9

Table 2 shows the mean computation time on a SUN 3/110 consumed by various heuristics for each sample. We see that the algorithm APRTF is a little more time-consuming. However, as we will see below, it gives a better solution than the others.

Table 3 B:SB

B: number of examples for which an algorithm is at least as good as UET
SB: number of examples for which an algorithm is strictly better than UET

n	20	30	40	50	60	70
APRTF	10:4	10:6	10:3	10:6	10:4	10:3
PRTF	5:3	6:5	4:3	3:3	2:2	2:2
UPRTF	10:4	10:7	10:4	10:7	10:4	10:4

Table 4 O:SO

O: number of examples for which an algorithm gives an optimal solution
SO: number of examples for which an algorithm gives an optimal solution but UET does not

n	20	30	40	50	60	70
APRTF	9:4	5:2	4:2	2:0	1:0	0:0
PRTF	5:3	4:1	2:1	0:0	0:0	0:0
UPRTF	9:4	6:3	4:2	2:0	1:0	0:0

If we compare the heuristics proposed in this paper with respectively UET and BAB, we obtain Table 3 and 4. From these tables, we see that APRTF always is better than UET. Although PRTF does not, but it gives sometimes a better solution than the one given by APRTF. Consequently, using UPRTF or simply APRTF seems to be the best way if one wants to obtain

a solution close to an optimal one.

5. Conclusion

We provided a necessary and sufficient condition which can be considered as a priority rule. On the basis of this condition, we have defined a new dominant subset, called F-active subset, containing all the optimal schedules. This reduces the set of schedules to be considered and is useful in enumerative optimal algorithm to prune the search tree.

Using the condition we have proved, we also have proposed some "good" heuristic algorithms which construct a F-active schedule. The experiments show that the algorithms proposed provide a better solution than the traditional priority rules. These heuristics can also be used to provide a good initial sequence in a BAB algorithm.

Although the result was obtained for the single machine scheduling problem, it can also be integrated into a flow-shop or a job-shop scheduling by considering the completion time of an operation as the release date of the next operation of the same job. PRTF function can also be used in an expert system as a priority rule.

Appendix: Analytic analysis of PRTF function

If we compare the priority of two jobs i and j , we can obtain some interesting properties (see Fig.4 and 5, in which we assume arbitrarily that $r_i \leq r_j$). Fig.4 and Fig.5 correspond respectively to the case $p_i > p_j$ and the case $p_i \leq p_j$.

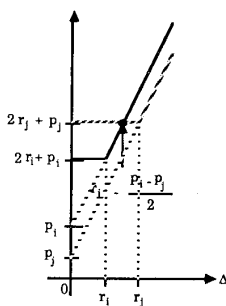


Fig. 4 case $p_i > p_j$

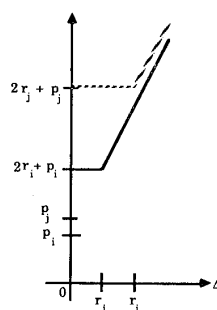


Fig.5 case $p_i \leq p_j$

In the case $p_i > p_j$ (Fig. 4), in the first period, job i has a higher priority than job j until $r_j - \frac{p_i - p_j}{2}$. From that moment, j always has a higher priority than job i .

In the case $p_i \leq p_j$ (Fig.5), job i always has a higher priority than job j .

We can then consider that, in each case, during the first period, PRTF is equivalent to a mixture of FIFO and SPT priority rules, where the weight of the FIFO rule is greater than SPT's. For the next period, it is equivalent to SPT. The PRTF priority rule is in fact a switcher from a "FIFO+SPT" rule to a SPT rule.

REFERENCES:

- [1] **K.R. BAKER**
Introduction to Sequencing and Scheduling
John & Wiley, 1974
- [2] **L. BIANCO, S. RICCIARDELLI**
"Scheduling of a Single Machine to Minimize Total Weighted Completion Time subject to Release Dates"
Nav. Res. Logist. Quart. Vol. 29, n° 1, pp. 151-167, 1982
- [3] **S. CHAND, H. SCHNEEBERGER**
"A Note on the Single Machine Scheduling Problem with Minimum Weighted Completion Time and Maximum Allowable Tardiness"
Naval. Res. Logist. Quart. vol. 33, pp. 551-557, 1986
- [4] **R. CHANDRA**
"On $n/1/\bar{F}$ Dynamic Deterministic Systems"
Nav. Res. Logist. Quart. vol. 26, pp. 537-544, 1979
- [5] **R.W. CONWAY, W.C. MAXWELL, L.W. MILLER**
Theory of Scheduling
Addison-Wesley, 1967
- [6] **J.S. DEOGUN**
"On Scheduling with Ready Times to Minimize Mean Flow Time"
The Computer Journal, Vol. 26, n° 4, pp. 320-328, 1983
- [7] **M.I. DESSOUKY, J.S. DEOGUN**
"Sequencing Jobs with Unequal Ready Times to Minimize Mean Flow Time"
SIAM. J. Comput. Vol. 10, n° 1, pp. 192-202, February 1981
- [8] **S.K. GUPTA, J. KYPARISIS**
"Single Machine Scheduling Research"
OMEGA Int. J. Mgmt. Sci. vol. 15, n° 3, pp. 207-227, 1987
- [9] **A.M.A. HARIRI, C.N. POTTS**
"An Algorithm for Single Machine Sequencing with Release Dates to Minimize Total Weighted Completion Time"
Disc. Appl. Math. n° 5, pp. 99-109, 1983
- [10] **J.K. LENSTRA, A.H.G. RINNOOY KAN, P. BRUCKER**
"Complexity of Machine Scheduling Problems"
Ann. Discrete. Math. n° 4, pp. 281-300, 1977
- [11] **S.S. PANWALKAR, W. ISKANDER**
"A Survey of Scheduling Rules"
Operations Research, Vol. 25, n° 1, pp. 45-61, January-February 1977
- [12] **M.E. POSNER**
"A Sequencing Problem with Release Dates and Clustered Jobs"
Man. Sci. Vol. 32, n° 6, pp. June 1986
- [13] **A.H.G. RINNOOY KAN**
"Machine Sequencing Problems: Classification, Complexity and Computation"
Nijhoff, The Hague, 1976
- [14] **W.E. SMITH**
"Various Optimizers for Single Stage Production"
Nav. Res. Logist. Quart. Vol. 3, pp. 59-66, 1956